

MASTER PROJECT HANDBOOK



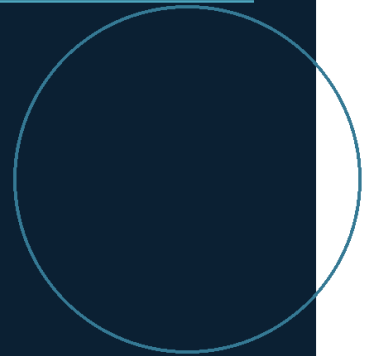
Time Irreversibility in Cell Motility

Biophysics • Python • Scientific Computing • Reproducible Research

Complete project record through the end of Phase 4

exact-lag correction • common-motion analysis • co-moving sensitivity branch

Repository main e0e0909 • README synchronized • Phase 5 handoff



Prepared for Abolfazl Pourhashem

Repository snapshot inspected 2 August 2026

Document purpose and evidence standard

This handbook remains the same cumulative project record established in the earlier edition. The Phase 1–3 material is preserved unless later validation revealed a conflict. This update adds the exact-lag correction, Phase 4 temporal-gap audit, common-motion analysis, co-moving sensitivity representation, expanded scientific tests, Git/GitHub history, README synchronization, and the exact handoff into Phase 5.

Evidence hierarchy used to build this handbook

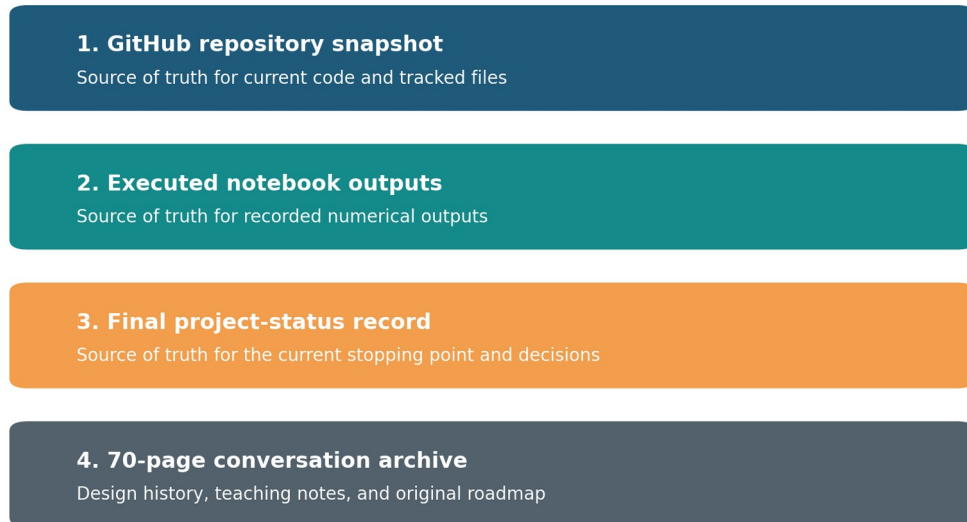


Figure 1. Evidence hierarchy used to reconcile project history, notebook outputs, and the live repository.

Evidence labels used throughout

REPOSITORY-VERIFIED means the item exists in the current GitHub main branch. NOTEBOOK-OUTPUT VERIFIED means the value is stored in an executed notebook output. PROJECT-RECORD REPORTED means it is documented in the final status record but is not present in the public repository. PLANNED means it has not yet been executed.

Project Handoff Brief — Validated Through Phase 4

Phase 4 is complete. Exact physical lags are enforced; raw and co-moving trajectory/step representations are validated. The temporary phase-4-drift branch was merged via PR #1 and deleted. README is synchronized. Next: Phase 5 density estimation and displacement-inversion asymmetry, raw vs co-moving.

Item	Current verified state
Repository	GitHub main e0e0909 (README updated through Phase 4); PR #1 merged; temporary Phase 4 branch deleted.
Dataset	CTC Fluo-C2DL-MSD sequence 01; 48 frames; 0.3 $\mu\text{m}/\text{pixel}$; 20 min/frame.
Clean trajectory table	474 observations, 35 tracks; cell_id/frame/t_min/x_um/y_um.
Exact-lag support	$\tau=1$: 433 steps / 35 cells; $\tau=2$: 398 / 32; $\tau=4$: 343 / 26; total 1,174 exact steps.
Phase 4 finding	Population-shared translation is strongest in negative x; technical vs biological origin is unresolved.
Sensitivity branch	Raw retained; co-moving coordinates subtract cumulative frame-wise population-median motion.
Next step	Phase 5: estimate $p(\Delta)$, compare $p(\Delta)$ vs $p(-\Delta)$, quantify uncertainty, raw vs co-moving.

Project workflow: validated through Phase 4



Key Phase 4 decision: retain both laboratory-frame (raw) and co-moving representations; do not label the shared component as microscope drift without independent evidence.

Figure 2. End-to-end project pipeline updated through Phase 4; Phase 5 is the next active analysis.

Repository link: <https://github.com/AbolDpirate/cell-irreversibility> | current verified main: e0e0909

[↑ Back to Contents](#)

Contents

Every entry below is an internal hyperlink. The document uses static links so navigation works without updating Word fields.

1. Scientific Motivation and Scope

- [1.1 The biological system](#)
- [1.2 Time reversal and non-equilibrium reasoning](#)
- [1.3 The updated interpretation of the original metric](#)

2. Roadmap: Original Plan vs. Executed Work

- [2.1 Original phases 0–11](#)
- [2.2 Updated phase structure](#)

3. Reproducible Project Architecture

- [3.1 Repository snapshot](#)
- [3.2 Conda environment](#)
- [3.3 Python packages, modules, notebooks, and tests](#)
- [3.4 Git and ignored artifacts](#)

4. Dataset and Image-Tracking Workflow

- [4.1 Dataset selection and metadata](#)
- [4.2 Fiji calibration](#)
- [4.3 TrackMate detection and tracking](#)
- [4.4 Raw CSV export](#)

5. Phase 2B – Cleaning the TrackMate Spots Table

- [5.1 Raw table anatomy](#)
- [5.2 Exact cleaning code](#)
- [5.3 Verified outputs](#)
- [5.4 Python learning module](#)

6. Mathematics of Trajectories and τ -Lagged Steps

- [6.1 Trajectory and displacement definitions](#)
- [6.2 Units and lag selection](#)
- [6.3 What a one-step distribution can and cannot prove](#)

7. The Reusable Phase 3 Module: `src/steps.py`

- [7.1 Current exact-frame source listing](#)
- [7.2 `validate\_tracks` and data invariants](#)
- [7.3 `compute\_steps\_for\_tau`: exact physical lags](#)
- [7.4 Multiple lags and summaries](#)
- [7.5 Python concepts learned from the correction](#)

8. Phase 3 Notebook and Corrected Numerical Baseline

- [8.1 Notebook rebuild and lag-audit logic](#)
- [8.2 Corrected step-support summary](#)
- [8.3 Corrected descriptive statistics](#)
- [8.4 Directional diagnostic after correction](#)
- [8.5 Saved raw-coordinate artifacts](#)

9. Phase 4 Common-Motion Analysis and Co-Moving Coordinates

- [9.1 Temporal-gap and exact-lag audit](#)
- [9.2 Robust frame-wise common-motion estimator](#)
- [9.3 Co-moving coordinate construction and validation](#)
- [9.4 Interpretation, limitations, and the image-registration decision](#)

10. Testing, Git History, and Reproducibility Audit

[10.1 Scientific test coverage](#)

[10.2 Git/GitHub workflow and documentation state](#)

[10.3 Remaining reproducibility gaps](#)

11. Exact Current Status and Phase 5 Roadmap

[11.1 Completed through Phase 4](#)

[11.2 Phase 5: density estimation and single-step asymmetry](#)

[11.3 Later: sequence-level temporal irreversibility](#)

[11.4 Uncertainty, controls, robustness, and thermodynamic boundary](#)

12. Resume Protocol for Phase 5

[12.1 Start every work session](#)

[12.2 Rebuild and verify raw + co-moving inputs](#)

[12.3 Begin Phase 5 on a dedicated branch](#)

[12.4 Commit, merge, and documentation checkpoint](#)

Appendix A. Full Supporting Files

Appendix B. Data Dictionaries and Artifact Ledger

Appendix C. Python and Biophysics Glossary

Appendix D. Ready-to-Paste New-Chat Handoff Prompt

Appendix E. References and Source Snapshot

1. Scientific Motivation and Scope

1.1 The biological system

Living cells are open, active systems. They consume chemical free energy, remodel their cytoskeleton, generate forces, change shape, and migrate. Those processes motivate a non-equilibrium analysis: can the recorded motion itself reveal statistical signatures that distinguish the observed direction of time from a reversed description?

- The observable is a two-dimensional cell-centroid trajectory extracted from time-lapse microscopy.
- The project is in silico: no new wet-lab experiment is required for the current stage.
- The analysis must distinguish biological motion from tracking artifacts and image/stage drift.
- Any entropy-production interpretation must be stated as a bound or coarse-grained proxy under explicit assumptions—not as the full thermodynamic dissipation of a cell.

1.2 Time reversal and non-equilibrium reasoning

At equilibrium, detailed balance implies a symmetry between appropriately defined forward and reverse path statistics. Away from equilibrium, the logarithm of a forward-path probability divided by its reverse-path probability is a central stochastic-thermodynamic quantity. The word path is crucial: a time-ordered sequence generally contains more information than a single displacement.

$$\text{Path-level irreversibility: } \log [P(\text{path}) / P(\text{time-reversed path})]$$

Interpretation rule

A difference between forward and reversed path probabilities can support time-reversal symmetry breaking. A difference between $p(\Delta)$ and $p(-\Delta)$ only establishes inversion asymmetry of single displacements unless extra assumptions connect it to path irreversibility.

1.3 The updated interpretation of the original metric

The original project plan proposed an index of the form below. It remains useful as an exploratory displacement-asymmetry statistic, but it should not automatically be labeled a complete temporal-irreversibility measure.

$$I\tau = \langle \log [p(\Delta\tau) / p(-\Delta\tau)] \rangle$$

Analysis level	What it directly measures	Appropriate interpretation
Single displacement	Whether (dx, dy) and $(-dx, -dy)$ have the same distribution	Directional bias / displacement inversion asymmetry
Two or more ordered steps	Whether short forward sequences resemble reversed sequences	Temporal ordering asymmetry
Full or sufficiently rich path statistics	Forward vs. reversed path probability ratio	Time irreversibility; possible entropy-production bound under assumptions

[↑ Back to Contents](#)

2. Roadmap: Original Plan vs. Executed Work

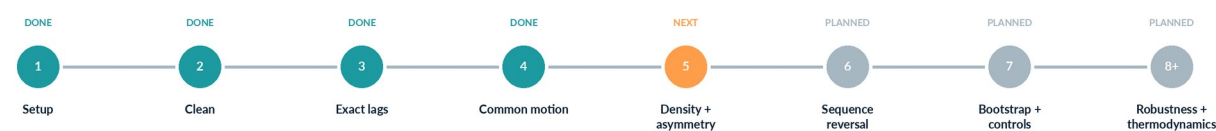
2.1 Original phases 0–11

Original phase	Purpose	Status at this snapshot
0	Conceptual orientation: equilibrium, time reversal, KDE, bootstrap, entropy production, TUR	Conceptual orientation completed; interpretation boundary retained
1	Environment and project skeleton	Completed and reproducibility-audited
2	Data import / tracking / standard trajectory CSV	Completed for sequence 01
3	Preprocessing and displacement steps	Completed; exact-lag bug discovered and

Original phase	Purpose	Status at this snapshot
		corrected
4	Density estimation	NEXT: Phase 5 density estimation on raw + co-moving exact steps
5	Irreversibility metric + bootstrap	Planned after density estimator is validated
6	Time-reversal and shuffle controls	Planned: sequence reversal, reversible surrogates, sign controls
7	Robustness	Planned: min10, lag grid, bandwidth/binning, sequence 02, tracking sensitivity
8	Entropy-production/TUR extension	Deferred until validated sequence-level metric
9	Publication figures	Planned after analysis stabilizes
10	README, Methods, one-page report	README synchronized through Phase 4; handbook updated in this document
11	Optional machine learning	Optional; not required for main scientific claim

2.2 Updated phase structure

Updated research roadmap



Phase 4 is closed. Phase 5 begins probability-density estimation on both raw and co-moving exact-lag steps.

Figure 3. Updated project phases after separating temporal-gap/common-motion validation from density and irreversibility analysis.

The updated roadmap treats temporal-gap validation and common-motion assessment as an independent gate before density estimation. This prevents a microscope/registration artifact from being mistaken for biological directionality while also preventing genuine collective migration from being removed without evidence.

[↑ Back to Contents](#)

3. Reproducible Project Architecture

3.1 Repository snapshot

The live GitHub repository was re-checked on 2 August 2026 after the Phase 4 pull request, branch cleanup, and README update. The current validated source-of-truth commit is e0e0909 (“Update README through Phase 4”). Commit history includes d0bf5e5 (merge PR #1), 37e5f31 (Phase 4 common-motion correction), 8be48d8 (exact-lag fix), and 683f487 (Phase 3 reproducibility audit). The branches page shows only main; the temporary phase-4-drift branch has been deleted after merge. The repository tree below is the current tracked structure; ignored biological data and generated local analysis products are intentionally absent.

Repository-verified tree

```
cell-irreversibility/  
├── .gitattributes  
├── .gitignore  
├── README.md  
├── env.yml  
└── envs/
```

```

├── environment-history.yml
├── explicit-win-64.txt
├── data/
│   ├── .keep                # biological/local outputs are ignored
├── figures/                  # generated figures ignored except .keep
├── instructions/
│   └── Cell_Motility_Irreversibility_Master_Handbook.pdf
├── notebooks/
│   ├── 02_clean_spots_MSC01.ipynb
│   ├── 03_compute_steps_MSC01.ipynb
│   ├── 04_drift_validation_MSC01.ipynb
│   └── _path.py
├── src/
│   ├── __init__.py
│   ├── io.py
│   ├── steps.py
│   ├── density.py
│   ├── metrics.py
│   └── plots.py
├── tests/
│   ├── conftest.py
│   ├── test_imports.py
│   └── test_steps.py

```

Important repository fact

The CSV inputs and outputs are intentionally ignored by .gitignore and are not present in the public repository. The figures directory is also absent from the tracked snapshot because empty ignored directories are not committed.

3.2 Conda environment

Repository file: env.yml

```

name: cell-irreversibility
channels:
  - defaults
dependencies:
  - python=3.11
  - matplotlib
  - pytest
  - numpy
  - pandas
  - scikit-learn
  - jupyterlab
  - ipykernel

```

The environment name is cell-irreversibility and the declared Python version is 3.11. The environment includes pandas for tables, NumPy for arrays and numerical work, Matplotlib for plots, scikit-learn for later statistical/ML extensions, JupyterLab and ipykernel for notebooks, and pytest for tests.

Environment creation and kernel registration

```

conda env create -f env.yml
conda activate cell-irreversibility
python -m ipykernel install --user --name cell-irreversibility --display-name "Python (cell-irreversibility)"

```

3.3 Python packages, modules, notebooks, and tests

Component	Role	Current state
src/steps.py	Validated exact physical-lag displacement construction	Implemented; exact-frame self-merge; duplicate cell-frame validation
src/io.py	Input/validation helpers	Placeholder / future expansion
src/density.py	Histogram/KDE density estimation	Placeholder; primary Phase 5 implementation target
src/metrics.py	Asymmetry, controls, bootstrap	Placeholder; Phase 5+
src/plots.py	Reusable plotting helpers	Placeholder; Phase 5+
notebooks/02...	TrackMate CSV cleaning	Executed historical workflow
notebooks/03...	Exact-lag multi-tau step computation	Executed; corrected baseline stored
notebooks/04...	Temporal-gap audit, common motion, co-	Executed; Phase 4 validation passed

Component	Role	Current state
	moving coordinates	
tests/test_imports.py	Import smoke test	Implemented
tests/test_steps.py	Scientific unit/regression tests	Implemented; 8 tests in file; total suite = 9 passed

A module is a .py file. A package is a directory that Python treats as a collection of modules; src/ __init__.py marks src as a package. A notebook is an interactive execution document. The project uses notebooks for transparent, stepwise analysis and src/ for reusable logic.

3.4 Git and ignored artifacts

Repository file: .gitignore

```
__pycache__/
.ipynb_checkpoints/
.pytest_cache/
.env
.venv
*.pyc
.DS_Store
.vscode/

# Local datasets and generated analysis outputs
data/*
!data/.keep
!data/README.md

# Generated figures
figures/*
!figures/.keep

# Large media files
*.mp4
```

Ignoring data can be appropriate when raw images or derived tables are large, licensed, or easily regenerated. However, reproducibility then requires a documented download step, checksums or dataset version, and executable scripts that recreate every derived artifact. At present, the public repository does not contain enough data artifacts to run the notebooks end-to-end without the local working directory.

[↑ Back to Contents](#)

4. Dataset and Image-Tracking Workflow

4.1 Dataset selection and metadata

Field	Recorded value
Dataset	Fluo-C2DL-MSC from the Cell Tracking Challenge
Sequence	01
Imaging	2D fluorescence time-lapse
Biological material	Rat mesenchymal stem cells on a flat polyacrylamide substrate (official CTC dataset description).
Frames	48
Pixel size	0.3 μm /pixel
Frame interval	20 min/frame

METADATA CORRECTION

Earlier project notes described human MSCs. The authoritative Cell Tracking Challenge dataset page identifies Fluo-C2DL-MSC as rat mesenchymal stem cells on a flat polyacrylamide substrate. This handbook therefore uses the official CTC description; publication Methods should cite the CTC dataset source directly.

4.2 Fiji calibration

1. Open sequence 01 as an image sequence/stack in Fiji.
2. Convert the stack interpretation to Channels = 1, Slices = 1, Frames = 48.
3. Set pixel width and height to 0.3 μm .
4. Set the frame interval to 20 minutes.
5. Confirm that TrackMate reads spatial and temporal calibration before detection.

4.3 TrackMate detection and tracking



No additional spot filters • No track filters • Median filter ON • Sub-pixel localization ON

Figure 4. Recorded Fiji/TrackMate workflow and parameters.

Stage	Parameter	Value
Detector	Algorithm	LoG detector
Detector	Estimated object diameter	28 μm
Detector	Quality threshold	2
Detector	Median filter	Enabled
Detector	Sub-pixel localization	Enabled
Spot filters	Additional filters	None
Tracker	Algorithm	Simple LAP tracker
Tracker	Linking max distance	8 μm
Tracker	Gap-closing max distance	15 μm
Tracker	Maximum frame gap	2 frames
Track filters	Additional filters	None

The Laplacian-of-Gaussian detector identifies approximately blob-like bright objects at a scale related to the estimated diameter. The Simple LAP tracker links detections across frames by solving an assignment problem subject to distance and gap-closing constraints. These parameters are not universal biological constants; they are analysis choices that should eventually be validated against visual inspection or reference annotations.

4.4 Raw CSV export

The final raw table was exported from TrackMate's Spots table rather than the MotilityLab export. Each row represents one detection at one time point and contains trajectory identifiers, coordinates, time, quality, radius, and intensity features.

Locally reported raw input path (not present in GitHub)

```
data/raw/Fluo-C2DL-MS01/MS01_allspots.csv
```

5. Phase 2B — Cleaning the TrackMate Spots Table

5.1 Raw table anatomy

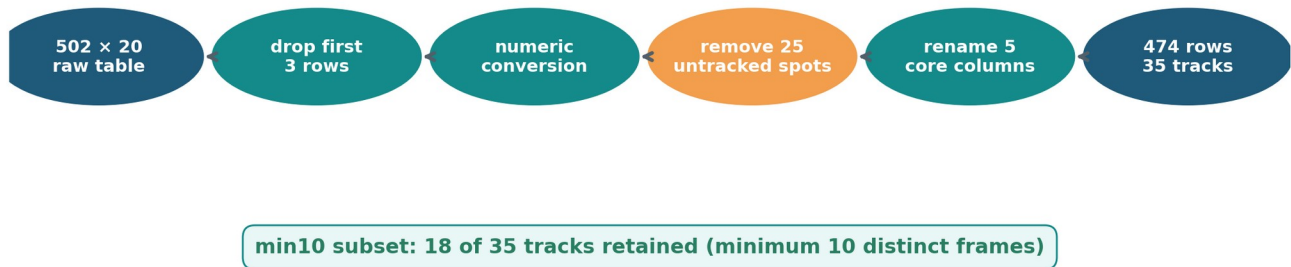


Figure 5. Data reduction from raw TrackMate export to canonical trajectories.

Checkpoint	Verified output
Raw pandas table	502 rows × 20 columns
After removing 3 metadata/unit rows	499 rows × 20 columns
Rows with missing TRACK_ID	25
Tracked detections retained	474 rows
Unique tracks	35
Track length distribution	mean 13.54; median 10; min 2; max 48 frames
min10 subset	18 of 35 tracks

The three first rows were not observations: TrackMate embedded labels and units inside the table. Their presence forced many numeric columns to be read as object/string dtype. Cleaning therefore required both row removal and explicit numeric conversion.

5.2 Exact cleaning code

Reconstructed cell order from the repository notebook

```

# Cell 1 – Load the raw TrackMate Spots CSV
import pandas as pd

raw_path = "../data/raw/Fluo-C2DL-MS01/MS01_allspots.csv"
df = pd.read_csv(raw_path)

df.shape, df.columns
print(df.head) # Current repository cell prints the method object, not df.head()

# Cell 2 – Remove TrackMate's three non-data header/unit rows
spots = df.iloc[3:].copy()
print(spots.shape)
spots.head()

# Cell 3 – Convert important columns from object/string to numeric types
num_cols = [
    "ID", "TRACK_ID", "QUALITY",
    "POSITION_X", "POSITION_Y", "POSITION_Z",
    "POSITION_T", "FRAME", "RADIUS", "VISIBILITY"

```

```

]

for c in num_cols:
    spots[c] = pd.to_numeric(spots[c], errors="coerce")

spots[num_cols].dtypes

# Cell 4 – Count detections without a TrackMate TRACK_ID
spots["TRACK_ID"].isna().sum()

# Cell 5 – Keep only detections that belong to a trajectory
spots = spots[spots["TRACK_ID"].notna()].copy()
spots.shape

# Cell 6 – Select and rename the canonical trajectory columns
cols_map = {
    "TRACK_ID": "cell_id",
    "FRAME": "frame",
    "POSITION_T": "t_min",
    "POSITION_X": "x_um",
    "POSITION_Y": "y_um",
}

tracks = spots[list(cols_map.keys())].rename(columns=cols_map)
print(tracks.head())
tracks.dtypes

# Cell 7 – Sort and enforce integer identifiers/frame indices
tracks = tracks.sort_values(["cell_id", "frame"]).reset_index(drop=True)
tracks["cell_id"] = tracks["cell_id"].astype(int)
tracks["frame"] = tracks["frame"].astype(int)
print(tracks.dtypes)
tracks.head()

# Cell 8 – Measure track lengths
lengths = tracks.groupby("cell_id")["frame"].nunique()
lengths.describe()

# Cell 9 – Create the min10 subset
min_len = 10
good_ids = lengths[lengths >= min_len].index
tracks_filtered = tracks[tracks["cell_id"].isin(good_ids)].copy()
tracks_filtered = tracks_filtered.sort_values(["cell_id", "frame"]).reset_index(drop=True)
tracks_filtered["cell_id"].nunique(), tracks["cell_id"].nunique()

# Cell 10 – Intended output paths
out_path1 = "../data/MS01_tracks_clean.csv"
out_path2 = "../data/MS01_tracks_clean_min10.csv"

# NOTE: these two lines are commented in the current GitHub notebook.
# tracks.to_csv(out_path1, index=False)
# tracks_filtered.to_csv(out_path2, index=False)

```

Current notebook bug

The first cell contains `print(df.head)`, which prints a bound method object. The intended inspection call is `df.head()` or `print(df.head())`.

Current reproducibility gap

The two `to_csv` lines in the GitHub notebook are commented. A clean “Restart Kernel and Run All” will not recreate `MS01_tracks_clean.csv` or `MS01_tracks_clean_min10.csv` unless those lines are uncommented or another script generates the files.

5.3 Verified outputs

Canonical column	Meaning	Expected dtype	Unit
cell_id	TrackMate trajectory identifier	int	dimensionless
frame	Image frame index	int	frame
t_min	Acquisition time	float	minutes
x_um	x-coordinate of detected centroid	float	μm

Canonical column	Meaning	Expected dtype	Unit
y_um	y-coordinate of detected centroid	float	μm

The full clean table contained 474 rows. The min10 table retained only trajectories with at least 10 distinct frames. This threshold is an implemented convenience, not yet a scientifically optimized inclusion criterion. The earlier planning document mentioned a 30-frame threshold, but the executed notebook used 10.

5.4 Python learning module

Python object/concept	Project example	Why it was used
DataFrame	df, spots, tracks	A labeled two-dimensional table similar to a spreadsheet
Series	tracks["cell_id"]	One labeled column; supports vectorized operations
list	num_cols	Ordered, mutable collection of column names
dictionary	cols_map	Maps old column names to new names as key-value pairs
set	required columns in steps.py	Unordered unique values; supports set difference for validation
tuple	tracks.shape → (474, 5)	Fixed-size ordered return value; convenient for paired dimensions
for loop	for c in num_cols	Repeat numeric conversion for each selected column
boolean mask	spots["TRACK_ID"].notna()	Select rows satisfying a condition
copy()	df.iloc[3:].copy()	Create an independent object and avoid ambiguous chained assignment
NaN	missing TRACK_ID	Standard missing-value marker in pandas/NumPy

A list is ordered and mutable; a tuple is ordered but immutable; a set is unordered and stores unique hashable elements; a dictionary stores key-value mappings. The choice is semantic: num_cols must preserve an explicit sequence, cols_map must associate each old name with a new name, and REQUIRED_TRACK_COLS must support fast membership and set subtraction.

[↑ Back to Contents](#)

6. Mathematics of Trajectories and τ -Lagged Steps

6.1 Trajectory and displacement definitions

For one cell i , the trajectory is the ordered sequence $r_i(t) = (x_i(t), y_i(t))$. The analysis transforms absolute positions into increments over a selected lag τ .

$$\Delta r(t) = r(t + \tau) - r(t) = (x(t + \tau) - x(t), y(t + \tau) - y(t))$$

A trajectory and one τ -lagged displacement vector

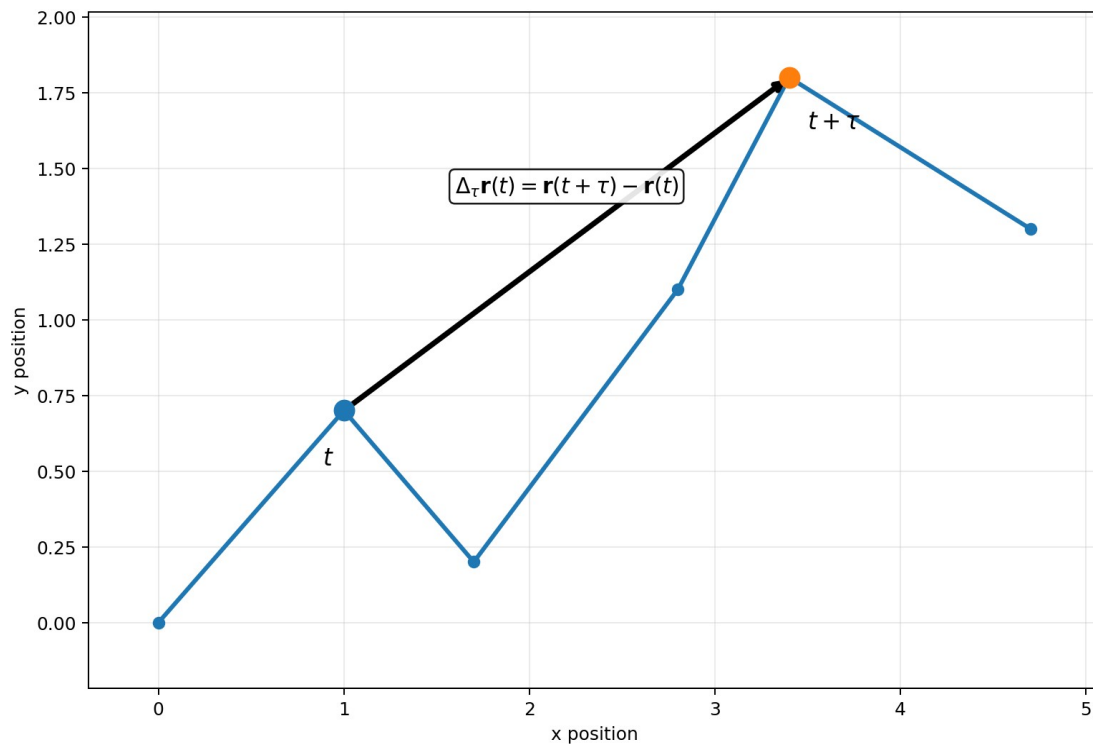


Figure 6. Geometric meaning of a τ -lagged displacement vector.

Absolute position depends on the arbitrary image origin. Displacements describe motion over a time interval and are therefore more directly related to velocity-like behavior, directional bias, persistence, and stochastic increments.

6.2 Units and lag selection

tau_frames	Nominal elapsed time	Reason retained
1	20 min	Maximum temporal resolution and largest sample
2	40 min	Intermediate scale
4	80 min	Longer-scale motion while retaining 26 contributing cells

The selected lags were 1, 2, and 4 frames. Larger lags such as 5 or 10 were avoided because short trajectories rapidly stop contributing, reducing both sample size and the number of independent cells.

6.3 What a one-step distribution can and cannot prove

A stationary active process may have a symmetric one-step increment distribution while still exhibiting irreversible ordering across multiple steps. For example, cyclic dynamics can look symmetric when projected onto a single increment but asymmetric in ordered pairs. Therefore, the project now separates displacement inversion asymmetry from proper temporal irreversibility.

Naming convention

Until sequence-level forward/reverse statistics are implemented, use “displacement inversion asymmetry” or “single-step directional asymmetry,” not an unqualified claim of entropy production or full time irreversibility.

[↑ Back to Contents](#)

7. The Reusable Phase 3 Module: src/steps.py

7.1 Current exact-frame source listing

The listing below is the current source-of-truth logic verified against GitHub main e0e0909. Comments are shortened only where that improves handbook readability; the algorithmic operations and public API match the repository implementation.

```
# src/steps.py
from __future__ import annotations
import pandas as pd

REQUIRED_TRACK_COLS = {"cell_id", "frame", "t_min", "x_um", "y_um"}

def validate_tracks(tracks: pd.DataFrame) -> None:
    """Validate the cleaned trajectory table and enforce one observation per cell/frame."""
    missing = REQUIRED_TRACK_COLS - set(tracks.columns)
    if missing:
        raise ValueError(f"tracks is missing required columns: {sorted(missing)}")

    duplicate_mask = tracks.duplicated(subset=["cell_id", "frame"], keep=False)
    if duplicate_mask.any():
        duplicate_pairs = (
            tracks.loc[duplicate_mask, ["cell_id", "frame"]]
            .drop_duplicates().head(5).to_dict(orient="records")
        )
        raise ValueError(
            "tracks contains duplicate cell_id-frame observations. "
            f"Examples: {duplicate_pairs}"
        )

def compute_steps_for_tau(tracks: pd.DataFrame, tau_frames: int) -> pd.DataFrame:
    """Compute displacement steps for an exact physical frame lag."""
    validate_tracks(tracks)
    if tau_frames < 1:
        raise ValueError("tau_frames must be >= 1")

    tracks_sorted = (tracks.sort_values(["cell_id", "frame"])
        .reset_index(drop=True).copy())

    starts = (tracks_sorted.rename(columns={
        "frame": "frame_start", "t_min": "t_start_min",
        "x_um": "x_start_um", "y_um": "y_start_um",
    })[["cell_id", "frame_start", "t_start_min", "x_start_um", "y_start_um"]].copy())
    starts["frame_end"] = starts["frame_start"] + int(tau_frames)

    ends = (tracks_sorted.rename(columns={
        "frame": "frame_end", "t_min": "t_end_min",
        "x_um": "x_end_um", "y_um": "y_end_um",
    })[["cell_id", "frame_end", "t_end_min", "x_end_um", "y_end_um"]].copy())

    steps = starts.merge(ends, on=["cell_id", "frame_end"],
        how="inner", validate="one_to_one")
    steps["dx_um"] = steps["x_end_um"] - steps["x_start_um"]
    steps["dy_um"] = steps["y_end_um"] - steps["y_start_um"]
    steps["tau_frames"] = int(tau_frames)
    steps["tau_min"] = steps["t_end_min"] - steps["t_start_min"]
    steps["cell_id"] = steps["cell_id"].astype(int)
    steps["frame_start"] = steps["frame_start"].astype(int)
    steps["frame_end"] = steps["frame_end"].astype(int)

    output_columns = ["cell_id", "frame_start", "t_start_min", "frame_end",
        "t_end_min", "dx_um", "dy_um", "tau_frames", "tau_min"]
    return (steps[output_columns].sort_values(["cell_id", "frame_start"])
        .reset_index(drop=True))

def compute_steps_multi_tau(tracks: pd.DataFrame, taus: list[int]) -> pd.DataFrame:
    validate_tracks(tracks)
    all_steps = []
    for tau in taus:
        all_steps.append(compute_steps_for_tau(tracks, tau_frames=tau))
    return pd.concat(all_steps, ignore_index=True)

def summary_by_tau(steps_all: pd.DataFrame) -> pd.DataFrame:
    return (steps_all.groupby("tau_frames")
        .agg(n_steps=("dx_um", "size"),
            n_cells=("cell_id", "nunique"),
            median_tau_min=("tau_min", "median"))
        .reset_index())
```

CRITICAL CHANGE

The authoritative implementation no longer uses `groupby(...).shift(-tau_frames)` to define physical lags. It computes the numeric target frame and performs an exact same-cell self-merge.

7.2 `validate_tracks` and data invariants

`validate_tracks` establishes invariants before any scientific calculation. A required-column set catches malformed inputs, and duplicate (`cell_id`, `frame`) observations are rejected because two positions for one cell at one frame would make a unique displacement endpoint impossible.

Check

Why it matters

Required columns	Prevents silent calculations on an incomplete schema.
Unique (cell_id, frame)	Guarantees a single spatial state per tracked cell per image frame.
Explicit error examples	Makes debugging data problems reproducible rather than silently dropping rows.

Python concept: `REQUIRED_TRACK_COLS` is a set. Sets store unique hashable values and make set difference efficient and readable: `required - observed` returns exactly the missing names. This is preferable here to a list because order is irrelevant and membership/uniqueness are the point.

7.3 `compute_steps_for_tau`: exact physical lags

Why row shift is not the same as physical frame lag

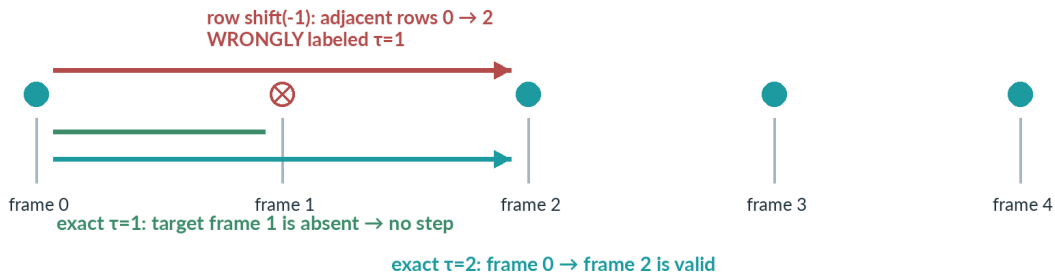


Figure 8. The exact-lag bug and the corrected self-merge logic. Adjacent rows are not necessarily adjacent physical frames.

The central mathematical requirement is $\text{frame_end} = \text{frame_start} + \tau_frames$. A start observation survives the inner merge only when the same cell has an observation at exactly that target frame. Missing intermediate frames are allowed for $\tau > 1$, but they can no longer change the meaning of the lag label.

Why `validate="one_to_one"` matters: pandas merge can otherwise multiply rows when either key is duplicated. The validation converts a potentially silent scientific error into an explicit exception.

HISTORICAL BUG VS. CORRECTED BASELINE

The old row-shift outputs contained 33 rows whose actual physical gap did not match their nominal τ label (6 at $\tau=1$, 12 at $\tau=2$, 15 at $\tau=4$). The net table-size reduction after rebuilding is only 13 rows because the exact-frame merge also recovers valid pairs that row-position shifting skipped. Therefore “33 mislabeled old rows” and “1,187 → 1,174 net rows” describe different accounting quantities.

7.4 Multiple lags and summaries

`compute_steps_multi_tau` loops over the requested lag list, calls the already-validated single-lag function, and concatenates the results. `summary_by_tau` then uses `groupby("tau_frames").agg(...)` to produce a compact support table. Reusing the single-lag function avoids having multiple independent definitions of a displacement in the codebase.

Canonical Phase 3 use

```
steps_all = compute_steps_multi_tau(tracks, taus=[1, 2, 4])
summary = summary_by_tau(steps_all)

# Authoritative support
#  $\tau=1$  -> 433 steps, 35 cells, median 20 min
#  $\tau=2$  -> 398 steps, 32 cells, median 40 min
#  $\tau=4$  -> 343 steps, 26 cells, median 80 min
```


7.5 Python concepts learned from the correction

Concept	What it is	Why used here
set	Unordered unique collection	Required-column validation and set difference.
DataFrame.rename	Returns a table with renamed labels	Creates explicit start/end schemas without mutating the original.
copy()	Creates an independent object	Avoids chained-assignment ambiguity and makes transformations explicit.
merge	Relational join between DataFrames	Matches a start observation to the same cell at the exact target frame.
validate="one_to_one"	Merge cardinality assertion	Prevents duplicate keys from silently multiplying scientific observations.
astype(int)	Explicit dtype conversion	Restores integer identity/frame semantics after table operations.
groupby + agg	Split-apply-combine pattern	Summarizes support by lag in a transparent way.
list[int]	Typed ordered collection	Carries the requested lags [1,2,4] and preserves iteration order.

[↑ Back to Contents](#)

8. Phase 3 Notebook and Corrected Numerical Baseline

8.1 Notebook rebuild and lag-audit logic

Notebook 03 remains the Phase 3 execution notebook, but after the audit it imports the corrected `src.steps` implementation. Notebook 04 independently rebuilt exact steps during the lag audit, so the same physical-lag definition was checked in two analysis contexts.

Minimal corrected notebook logic

```
taus = [1, 2, 4]
steps_all = compute_steps_multi_tau(tracks, taus=taus)
summary = summary_by_tau(steps_all)

# Independent audit
steps_all["actual_frame_gap"] = steps_all["frame_end"] - steps_all["frame_start"]
assert (steps_all["actual_frame_gap"] == steps_all["tau_frames"]).all()
```

Lag audit stage	$\tau=1$	$\tau=2$	$\tau=4$
Old row-shift total rows	439	404	344
Old rows with wrong physical gap	6	12	15
Exact-frame rebuilt rows	433	398	343
Exact-frame mismatches after rebuild	0	0	0

8.2 Corrected step-support summary

τ (frames)	Physical lag	n_steps	n_cells	median τ (min)
1	20 min	433	35	20.0
2	40 min	398	32	40.0
4	80 min	343	26	80.0

The combined exact-lag table contains 1,174 rows. The earlier row-shift table contained 1,187 rows. The difference of 13 is a net support change, not the count of all previously mislabeled rows; exact matching both removes invalid labels and can recover valid exact-frame pairs that a row offset fails to select.

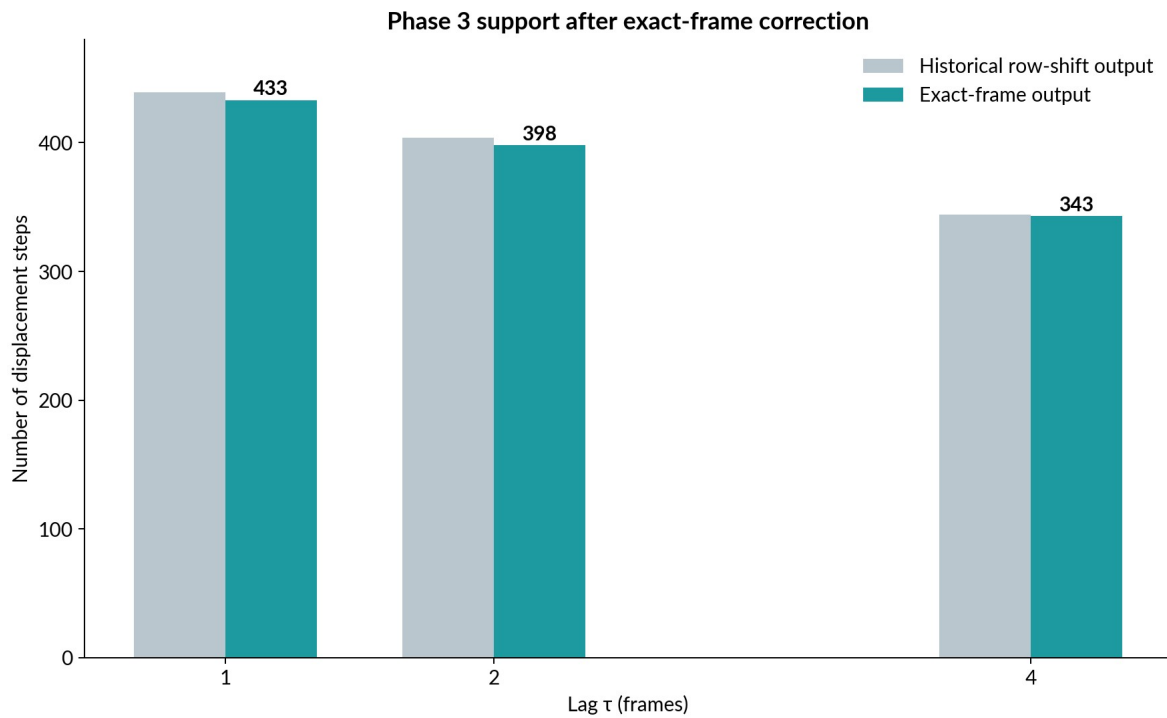


Figure 9. Historical row-shift support versus the corrected exact-frame support.

8.3 Corrected descriptive statistics

Statistic across all exact steps	dx_um	dy_um
count	1174	1174
mean	-0.958810	-0.342119
standard deviation	4.503926	4.911907
minimum	-24.842169	-34.010622
25th percentile	-2.901466	-1.447312
median	-0.763490	0.000000
75th percentile	1.155532	1.286550
maximum	17.578750	19.435225

These descriptive statistics summarize displacement observations, not independent biological replicates. Steps from the same track are temporally correlated, so later confidence intervals must resample at the track level rather than treating 1,174 rows as independent samples.

8.4 Directional diagnostic after correction

τ	mean dx (μm)	mean dy (μm)	median dx (μm)	median dy (μm)
1	-0.449440	-0.064475	-0.483247	0.000000
2	-0.885750	-0.273071	-1.043047	0.000000
4	-1.686608	-0.772733	-1.747158	0.000000

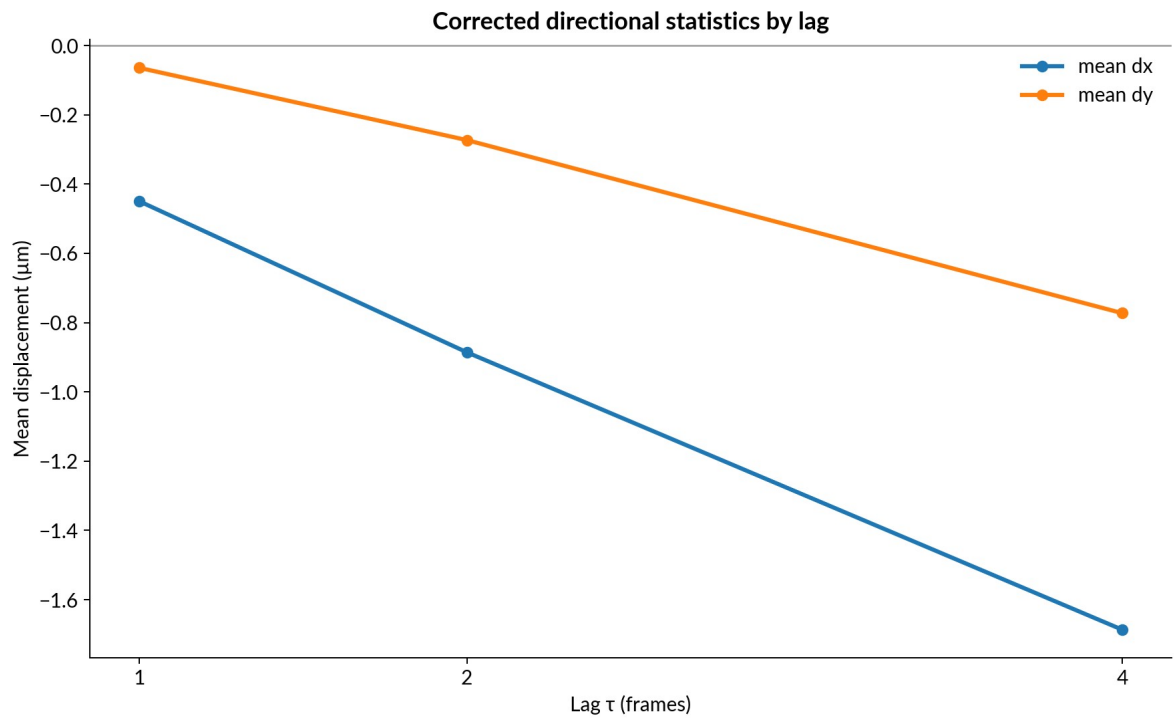


Figure 10. Corrected mean displacement by lag. Negative x grows in magnitude with lag, motivating the Phase 4 common-motion audit.

INTERPRETATION BOUNDARY
Approximate scaling of mean displacement with lag is consistent with a persistent directional component, but it does not identify whether the source is stage/sample motion, coordinated cell migration, or both. This observation motivated Phase 4; it is not by itself evidence of microscope drift or temporal irreversibility.

8.5 Saved raw-coordinate artifacts

Local artifact	Role	Git status
data/MSCO1_steps_tau124.csv	Combined exact raw-coordinate steps for $\tau=1,2,4$	Ignored/local
data/MSCO1_steps_tau1.csv	Exact $\tau=1$ raw steps	Ignored/local
data/MSCO1_steps_tau2.csv	Exact $\tau=2$ raw steps	Ignored/local
data/MSCO1_steps_tau4.csv	Exact $\tau=4$ raw steps	Ignored/local
data/MSCO1_steps_summary_tau124.csv	Per-lag support summary	Ignored/local

Because data/* is ignored, a fresh Git clone does not contain these biological/intermediate CSVs. Reproduction therefore requires either the documented local dataset or a future scripted acquisition/preparation workflow plus checksum verification.

[↑ Back to Contents](#)

9. Phase 4 Common-Motion Analysis and Co-Moving Coordinates

9.1 Temporal-gap and exact-lag audit

Phase 4 began with a temporal-gap audit rather than immediately subtracting “drift.” This changed the project in an important way: before interpreting any directional component, the code first had to prove that a label such as $\tau=1$ really represented one physical frame.

Observed within-track gap	Elapsed time	Number of transitions
1 frame	20 min	433
2 frames	40 min	6

- Duplicate (cell_id, frame) rows: 0.

- Frame/time calibration inconsistencies: 0.
- All 47 possible global consecutive-frame transitions had at least one exact $\tau=1$ cell step.
- After rebuilding with the corrected exact-frame implementation, total lag-label mismatches: 0.

This audit distinguishes two kinds of missingness. A cell can disappear for one intermediate frame, producing a within-track gap of two frames, while the movie as a whole can still have coverage for every global consecutive transition because other cells are present. That is exactly what happens here.

9.2 Robust frame-wise common-motion estimator

For each consecutive transition $t \rightarrow t+1$, the project pools all exact $\tau=1$ steps that start at t and uses the population median displacement as a robust common translation estimator:

$$c_x(t) = \text{median}_i[dx_i(t)], \quad c_y(t) = \text{median}_i[dy_i(t)]$$

$$\text{MAD}_x(t) = \text{median}_i |dx_i(t) - c_x(t)| \quad (\text{and analogously for } y)$$

The median is deliberately used instead of the mean because a few fast or irregularly tracked cells should not define the global translation. The unscaled median absolute deviation (MAD) quantifies within-transition dispersion without assuming Gaussian motion.

Quantity	Verified result
Cells per transition	minimum 6; median 9; maximum 12
Mean cells per transition	9.212766
Median of frame-wise median dx	-0.300000 $\mu\text{m}/\text{frame}$
Median of frame-wise median dy	0.000000 $\mu\text{m}/\text{frame}$
Final cumulative common dx	-16.746817 μm
Final cumulative common dy	+3.071543 μm
Transitions with negative median dx	0.6170 (61.7%)
Transitions with negative median dy	0.3404 (34.0%)
Median common-motion magnitude	0.724896 $\mu\text{m}/\text{frame}$
Median MAD dx	1.142182 μm
Median MAD dy	0.771806 μm

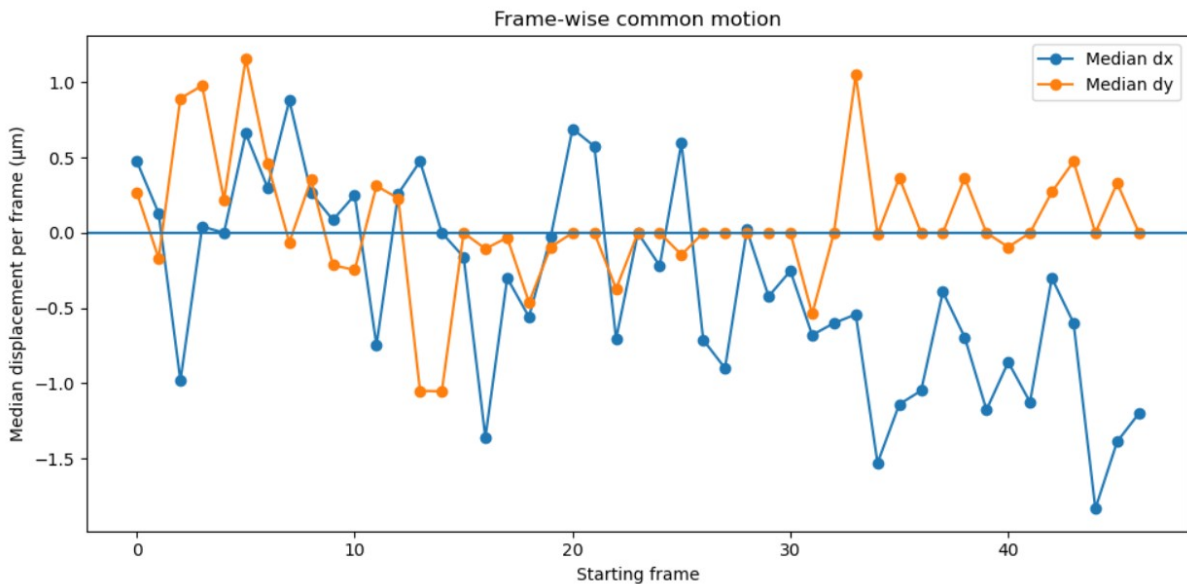


Figure 11. Frame-wise population-median motion. The x component becomes predominantly negative in later frames, but the signal is variable rather than a smooth constant translation.

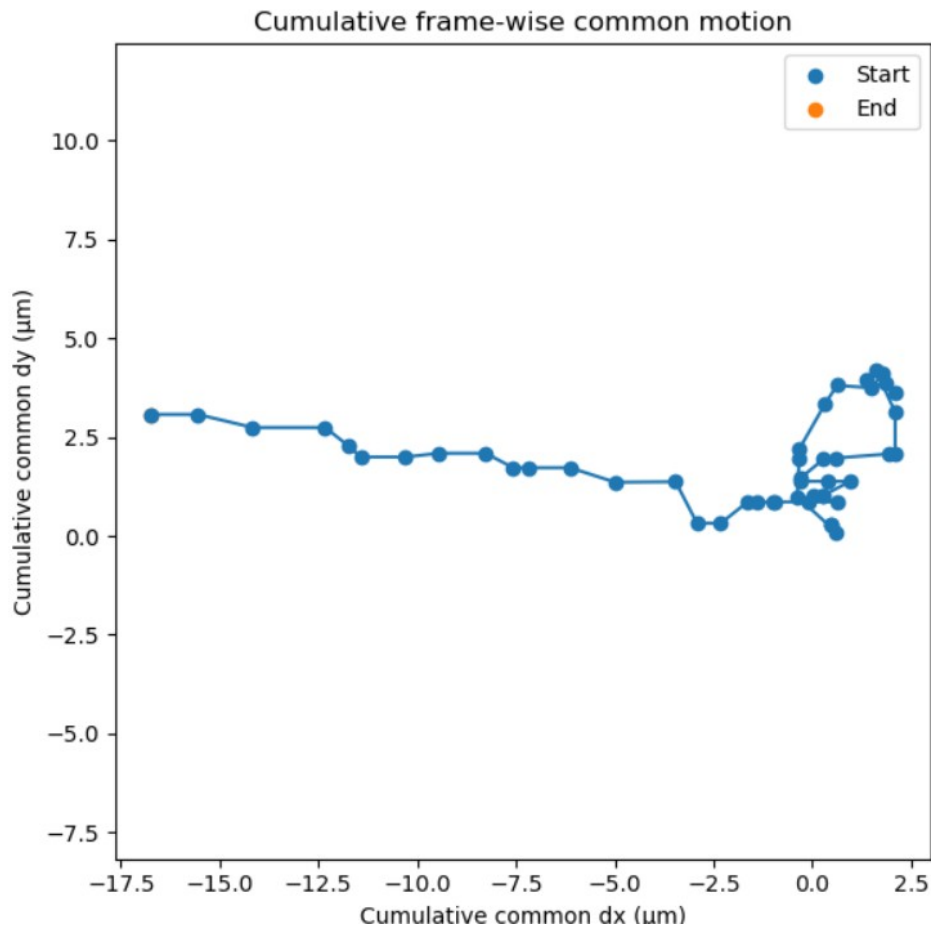


Figure 12. Cumulative frame-wise common motion. The net displacement is dominated by negative x and ends near (-16.75, +3.07) μm .

WHY THIS IS NOT AUTOMATICALLY “STAGE DRIFT”

Only about 61.7% of transitions have negative median dx, the frame-wise motion is non-stationary, and within-transition MAD is often comparable to or larger than the common-motion magnitude. The tracks demonstrate a shared directional component, but they do not uniquely identify its physical origin.

9.3 Co-moving coordinate construction and validation

The cumulative common-motion vector defines a co-moving coordinate frame. Let $c(t)=[c_x(t),c_y(t)]$. The cumulative translation is

$$C(0) = [0,0], \quad C(t) = \sum_{k=0}^{t-1} c(k)$$

and each position is transformed as

$$r_i^{\text{comoving}}(t) = r_i^{\text{raw}}(t) - C(t).$$

Core co-moving transformation

```
frame_correction = pd.DataFrame({
    "frame": [int(tracks["frame"].min())],
    "common_x_um": [0.0],
    "common_y_um": [0.0],
})

cumulative_part = pd.DataFrame({
    "frame": framewise_motion["frame_end"],
    "common_x_um": framewise_motion["median_dx_um"].cumsum(),
    "common_y_um": framewise_motion["median_dy_um"].cumsum(),
})

frame_correction = pd.concat([frame_correction, cumulative_part], ignore_index=True)

tracks_comoving = tracks.merge(frame_correction, on="frame", how="left", validate="many_to_one")
tracks_comoving["x_um_raw"] = tracks_comoving["x_um"]
tracks_comoving["y_um_raw"] = tracks_comoving["y_um"]
tracks_comoving["x_um"] = tracks_comoving["x_um"] - tracks_comoving["common_x_um"]
tracks_comoving["y_um"] = tracks_comoving["y_um"] - tracks_comoving["common_y_um"]
```

This is a coordinate transformation, not a new tracking algorithm. Every cell observed in the same frame receives the same translation. Cell identity, frame number, timestamp, and track topology therefore remain unchanged.

τ	Raw mean dx	Raw mean dy	Co-moving mean dx	Co-moving mean dy
1	-0.449440	-0.064475	-0.072486	-0.126184
2	-0.885750	-0.273071	-0.132785	-0.408888
4	-1.686608	-0.772733	-0.293470	-1.040118
τ	Raw median dx	Raw median dy	Co-moving median dx	Co-moving median dy
1	-0.483247	0.000000	0.000000	0.000000
2	-1.043047	0.000000	-0.142888	0.000000
4	-1.747158	0.000000	-0.315096	-0.271237

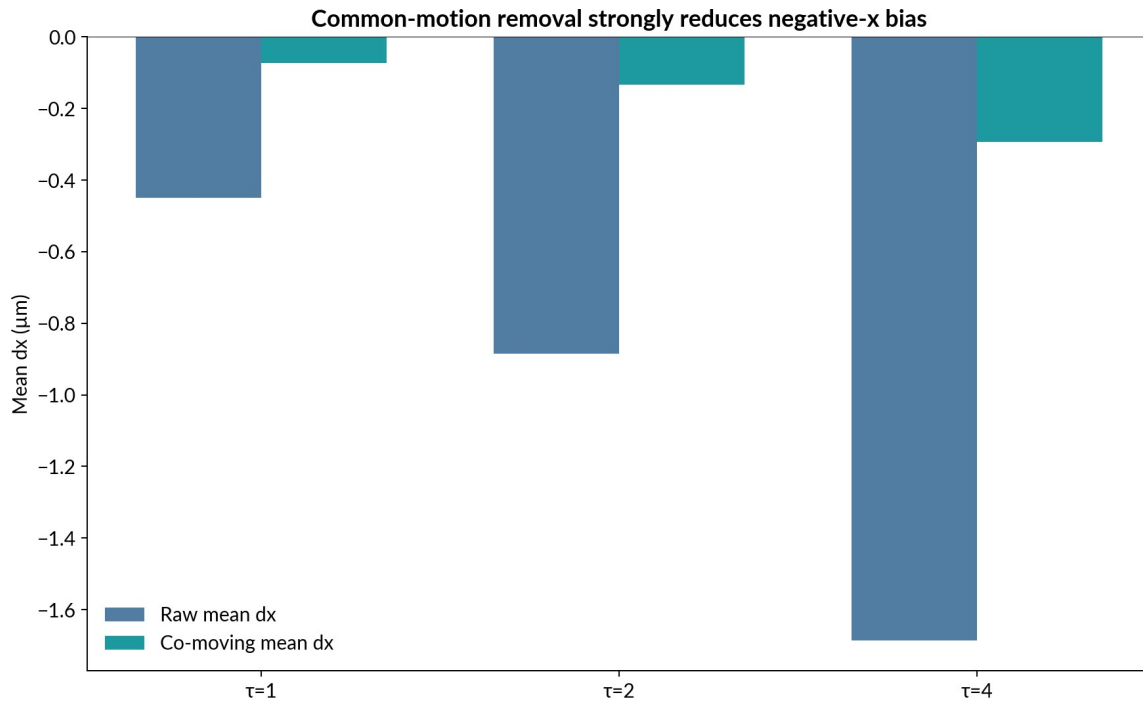


Figure 13. Removing the frame-wise median common translation strongly reduces the negative-x mean, especially at $\tau=1$.

At $\tau=1$, the median co-moving residual is zero by construction because the frame-wise median is exactly what was subtracted. The mean need not be zero because a median correction does not force the arithmetic mean to vanish. At $\tau=2$ and $\tau=4$, residual bias can remain because each long displacement combines several frame corrections and the set of contributing cells changes with the interval.

Validation gate	Result
Missing common_x/common_y corrections	0
Exact-lag mismatches after transformation	0
Co-moving step counts	{1: 433, 2: 398, 4: 343}
tracks output	MSC01_tracks_comoving.csv exists locally
steps output	MSC01_steps_comoving_tau124.csv exists locally
Final assertion block	Phase 4 validation: PASSED

9.4 Interpretation, limitations, and the image-registration decision

Image registration with phase cross-correlation (for example through scikit-image) was considered as an independent check. The idea was to estimate apparent frame-to-frame image translation and compare it with the cell-median motion.

The difficulty is biological identifiability: these fluorescence images are dominated by the cells themselves. If the cells genuinely migrate together, a registration algorithm can follow that biological motion and report it as “image motion.” Strong agreement with the trajectory median would therefore still not prove technical stage drift unless the registration is anchored to stationary background structure, fiducials, or acquisition metadata.

FINAL PHASE 4 DECISION Do not make image registration a blocking requirement. Preserve raw coordinates; construct the co-moving representation as a sensitivity analysis; carry both branches into Phase 5. Use “common-motion correction” or “co-moving coordinates,” not “definitive drift correction,” unless independent evidence later identifies a technical origin.	
Future observation	Defensible interpretation
Asymmetry strong in raw, absent in co-moving	Likely dominated by global directional translation; weak support for cell-relative irreversibility.
Asymmetry persists in both	More robust to common translation; still requires sequence-level reversal controls and uncertainty.
Asymmetry changes direction or grows after correction	Inspect support, track composition, and longer-lag y behavior before interpretation.
Results unstable across binning/bandwidth	Finite-sample density estimation dominates; do not report a definitive metric.

[↑ Back to Contents](#)

10. Testing, Git History, and Reproducibility Audit

10.1 Scientific test coverage

The test suite evolved from a single import smoke test to scientific unit/regression tests that encode the meaning of the displacement calculation. Current project output reports 9 tests passing in the validated environment.

Test	Scientific protection
test_imports	Package/module imports remain valid.
test_compute_steps_tau1	Hand-checkable $\tau=1$ within-cell displacements.
test_compute_steps_tau2	Direct frame 0→2 exact-lag calculation.
test_steps_do_not_cross_cell_boundaries	Prevents construction of a displacement between different cells.
test_invalid_tau_raises_error	Rejects physically/computationally invalid lag <1 .
test_missing_required_column_raises_error	Fails clearly when trajectory schema is incomplete.
test_multi_tau_summary	Checks concatenation and per-lag support.
test_missing_frame_is_not_mislabeled_as_tau1	Regression test for the row-shift bug: frames 0 and 2 produce no $\tau=1$ step but one valid $\tau=2$ step.
test_duplicate_cell_frame_raises_error	Rejects ambiguous duplicate observations.

This illustrates a central research-software lesson: code can be syntactically valid and still be scientifically wrong. The original shift-based implementation ran without errors; only a test that encoded the physical meaning of “one frame” could prevent the same bug from returning.

10.2 Git/GitHub workflow and documentation state

Phase 4 was developed on a temporary branch, reviewed through a GitHub pull request, merged into main, and then cleaned up. The repository was re-checked after the cleanup and README update.

Commit / state	Meaning
683f487	Validate Phase 3 reproducibility and audit project setup
8be48d8	Fix displacement steps to use exact frame lags
37e5f31	Validate exact lags and add common-motion correction
d0bf5e5	Merge pull request #1 from phase-4-drift into main
e0e0909	Update README through Phase 4 — current verified main at handbook update
Branches	Only main remains; phase-4-drift was deleted after merge.
Working-tree expectation	main synchronized with origin/main and clean before Phase 5 branch creation.

The README is now intentionally concise: it serves as the repository landing page, while this handbook is the detailed scientific, educational, and handoff record. This division prevents README from becoming a second, divergent handbook.

REPOSITORY DOCUMENTATION STATE GitHub main now reports Phase 4 complete and Phase 5 next, lists exact support 433/398/343, and uses the scientifically conservative common-motion/co-moving terminology. The tracked handbook PDF will become synchronized when this updated file replaces the older repository copy in a documentation-only commit.
--

10.3 Remaining reproducibility gaps

Remaining gap	Why it matters	Recommended action
---------------	----------------	--------------------

Biological data ignored	A fresh clone cannot execute notebooks directly.	Document acquisition/preparation plus checksums; optionally add a tiny synthetic fixture.
Notebook 02 historical metadata/runtime	Older execution metadata may not match the declared 3.11 environment.	Normalize/re-run when cleaning pipeline is next revisited.
density.py / metrics.py / plots.py placeholders	Phase 5 has not yet moved core analysis into tested reusable modules.	Implement incrementally with tests; avoid notebook-only final logic.
Image-based drift identification not done	Common motion remains physically ambiguous.	Keep optional robustness analysis; only claim technical drift with independent stationary reference.
min10 and sequence 02 not yet analyzed	Current inference may depend on track-duration selection or one movie.	Use in later robustness phase after Phase 5 estimator is stable.

[↑ Back to Contents](#)

11. Exact Current Status and Phase 5 Roadmap

11.1 Completed through Phase 4

- Environment, kernel, repository structure, and package versions validated.
- TrackMate extraction and cleaning completed for sequence 01.
- Exact $\tau=1,2,4$ displacement construction implemented and regression-tested.
- Temporal-gap audit completed; all rebuilt lag labels are exact.
- Frame-wise common motion quantified with median and MAD.
- Co-moving trajectories and exact-lag steps constructed, saved locally, and validated.
- Phase 4 branch merged through PR #1, temporary branch deleted, and README synchronized through Phase 4.

EXACT CURRENT STOPPING POINT

Do not redo Phase 4. Begin Phase 5 by loading the exact raw and co-moving step representations and building a tested density-estimation workflow.

11.2 Phase 5: density estimation and single-step asymmetry

Phase 5 is the first stage that estimates $p(\Delta)$ and $p(-\Delta)$. The analysis must be run in parallel for raw and co-moving steps and separately for each lag. The safest first estimator is a transparent 2D histogram on a symmetric (dx,dy) domain; KDE can then be used as a sensitivity analysis rather than as the only result.

For a single-step displacement Δ , define a carefully named displacement-inversion asymmetry quantity such as

$$I_{\tau}(\text{single-step}) = \langle \log [p_{\tau}(\Delta) / p_{\tau}(-\Delta)] \rangle.$$

NAMING RULE

Until ordered temporal sequences are analyzed, call this a displacement-inversion or single-step asymmetry statistic. Do not present it as the final trajectory-level irreversibility or a direct entropy-production rate.

Step	Implementation target
1	Load exact raw steps and exact co-moving steps.
2	For each τ and branch, define one common symmetric dx/dy support so Δ and $-\Delta$ are evaluated on identical domains.
3	Implement a transparent 2D histogram estimator in src/density.py; expose bins/support explicitly.
4	Visualize $p(\Delta)$, $p(-\Delta)$, probability support, and log-ratio maps; mask unsupported/near-zero regions.
5	Implement the single-step asymmetry statistic in src/metrics.py with explicit numerical safeguards.
6	Add unit tests using symmetric toy distributions (expected near zero) and intentionally biased distributions.
7	Bootstrap by cell track, not by individual step, and compare raw vs co-moving before deeper interpretation.

PHASE 5 STOP RULE

Do not report a final asymmetry value if it is unstable to binning/bandwidth, dominated by unsupported tails or pseudocount choices, or if raw and co-moving branches disagree without a mechanistic explanation.

11.3 Later: sequence-level temporal irreversibility

A sign inversion $\Delta \rightarrow -\Delta$ reverses a displacement vector in space but does not reverse temporal ordering. Full time reversal becomes clearer for an ordered block. For two consecutive steps,

$$\text{Forward block: } (\Delta_t, \Delta_{t+1}) \quad \text{Time-reversed block: } (-\Delta_{t+1}, -\Delta_t)$$

A later sequence-level statistic should compare the probability of ordered forward blocks with the probability of their correctly reversed blocks. That stage is where the project can more defensibly discuss temporal irreversibility, subject to finite-sample and coarse-graining limitations.

11.4 Uncertainty, controls, robustness, and thermodynamic boundary

Work package	Minimum requirement
Uncertainty	Cluster/bootstrap resampling at the cell-track level; report 95% intervals and effective number of tracks.
Estimator controls	Bin-grid and KDE-bandwidth sensitivity; support masks; pseudocount/smoothing sensitivity.
Null / symmetry controls	Sign randomization for single-step asymmetry; reversible surrogate; sequence reversal control for ordered blocks.
Biological robustness	Full vs min10; lag grid; sequence 01 vs sequence 02; tracking sensitivity when feasible.
Thermodynamic interpretation	Only connect path-probability ratios to entropy production under stated stochastic assumptions, time-reversal parity rules, and coarse-graining limitations.
Reporting	Record seeds, parameters, data provenance, exact commit, tests, figures, and a reproducible execution order.

The thermodynamic goal remains scientifically attractive, but it is intentionally downstream. A robust nonzero single-step asymmetry is evidence of directional statistical structure, not automatically a cellular entropy-production rate. A path-level likelihood ratio under an appropriate stochastic model is the stronger object needed for thermodynamic interpretation.

[↑ Back to Contents](#)

12. Resume Protocol for Phase 5

12.1 Start every work session

Start every work session from the repository root

```
conda activate cell-irreversibility
git switch main
git pull --ff-only origin main
git status --short --branch
python -m pytest -q
```

- Open the project root folder in VS Code, not only notebooks/.
- Select Python (cell-irreversibility) as the Jupyter kernel.
- Expect the test suite to report 9 passed before new Phase 5 code is written.
- Expect main to be synchronized with origin/main and clean.

12.2 Rebuild and verify raw + co-moving inputs

Confirm the ignored/local input files exist before assuming a fresh clone is runnable:

```
data/MS01_tracks_clean.csv
data/MS01_steps_tau124.csv
data/MS01_tracks_comoving.csv
data/MS01_steps_comoving_tau124.csv
```

Re-run the relevant validation cells if any file has been regenerated. The key invariants are exact counts {1:433, 2:398, 4:343}, zero lag mismatches, zero missing frame corrections, and unchanged 35-cell track identity in the raw coordinate table.

12.3 Begin Phase 5 on a dedicated branch

Suggested Phase 5 branch

```
git switch -c phase-5-distributions
# develop src/density.py + tests + a Phase 5 notebook
# keep raw and co-moving branches parallel
python -m pytest -q
```

Do not reimplement exact-lag construction inside the new notebook. Import the tested functions from `src/steps.py`. New density and metric logic should also move into reusable modules as soon as the exploratory definition is stable enough to test.

12.4 Commit, merge, and documentation checkpoint

Checkpoint pattern

```
git status --short
git add src tests notebooks README.md
git commit -m "Add raw and co-moving displacement distribution analysis"
git push -u origin phase-5-distributions
# open PR only after tests and notebook validation pass
```

After a validated Phase 5 merge, update this handbook by continuing the same document again: preserve validated Phase 1–4 sections, correct only conflicts, append the new scientific decisions/results, refresh the linked Contents entries, and replace the repository PDF in a documentation commit.

[↑ Back to Contents](#)

Appendix A. Full Supporting Files

A1. `env.yml`

```
name: cell-irreversibility
channels:
  - defaults
dependencies:
  - python=3.11
  - matplotlib
  - pytest
  - numpy
  - pandas
  - scikit-learn
  - jupyterlab
  - ipykernel
```

A2. `.gitignore`

```
__pycache__/
.ipynb_checkpoints/
.pytest_cache/
.env
.venv
*.pyc
.DS_Store
.vscode/

# Local datasets and generated analysis outputs
data/*
!data/.keep
!data/README.md

# Generated figures
figures/*
!figures/.keep

# Large media files
*.mp4
```

A3. `notebooks/_path.py`

```
import sys
from pathlib import Path
```

```
PROJECT_ROOT = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(PROJECT_ROOT))
```

A4. tests/conftest.py

```
import sys, os
ROOT = os.path.abspath(os.path.join(os.path.dirname(__file__), ".."))
if ROOT not in sys.path:
    sys.path.insert(0, ROOT)
```

A5. tests/test_imports.py

```
def test_imports():
    import src.io, src.steps, src.density, src.metrics, src.plots
```

A6. src/__init__.py

```
# makes src a package
```

A7. src/io.py

```
"""I/O placeholders: later we'll add read_tracks_csv() etc."""
```

A8. src/density.py

```
"""Density placeholders: later we'll add histogram2d / KDE."""
```

A9. src/metrics.py

```
"""Metrics placeholders: later we'll implement I_tau, controls, bootstrap."""
```

A10. src/plots.py

```
"""Plots placeholders: later we'll add plotting helpers."""
```

A11. Current src/steps.py – exact-frame implementation

```
# src/steps.py
from __future__ import annotations
import pandas as pd

REQUIRED_TRACK_COLS = {"cell_id", "frame", "t_min", "x_um", "y_um"}

def validate_tracks(tracks: pd.DataFrame) -> None:
    """Validate the cleaned trajectory table and enforce one observation per cell/frame."""
    missing = REQUIRED_TRACK_COLS - set(tracks.columns)
    if missing:
        raise ValueError(f"tracks is missing required columns: {sorted(missing)}")

    duplicate_mask = tracks.duplicated(subset=["cell_id", "frame"], keep=False)
    if duplicate_mask.any():
        duplicate_pairs = (
            tracks.loc[duplicate_mask, ["cell_id", "frame"]]
            .drop_duplicates().head(5).to_dict(orient="records")
        )
        raise ValueError(
            "tracks contains duplicate cell_id-frame observations. "
            f"Examples: {duplicate_pairs}"
        )

def compute_steps_for_tau(tracks: pd.DataFrame, tau_frames: int) -> pd.DataFrame:
    """Compute displacement steps for an exact physical frame lag."""
    validate_tracks(tracks)
    if tau_frames < 1:
        raise ValueError("tau_frames must be >= 1")

    tracks_sorted = (tracks.sort_values(["cell_id", "frame"])
        .reset_index(drop=True).copy())

    starts = (tracks_sorted.rename(columns={
        "frame": "frame_start", "t_min": "t_start_min",
        "x_um": "x_start_um", "y_um": "y_start_um",
    })[["cell_id", "frame_start", "t_start_min", "x_start_um", "y_start_um"]].copy())
    starts["frame_end"] = starts["frame_start"] + int(tau_frames)

    ends = (tracks_sorted.rename(columns={
        "frame": "frame_end", "t_min": "t_end_min",
        "x_um": "x_end_um", "y_um": "y_end_um",
    })[["cell_id", "frame_end", "t_end_min", "x_end_um", "y_end_um"]].copy())

    steps = starts.merge(ends, on=["cell_id", "frame_end"],
        how="inner", validate="one_to_one")
    steps["dx_um"] = steps["x_end_um"] - steps["x_start_um"]
    steps["dy_um"] = steps["y_end_um"] - steps["y_start_um"]
    steps["tau_frames"] = int(tau_frames)
    steps["tau_min"] = steps["t_end_min"] - steps["t_start_min"]
    steps["cell_id"] = steps["cell_id"].astype(int)
```

```

steps["frame_start"] = steps["frame_start"].astype(int)
steps["frame_end"] = steps["frame_end"].astype(int)

output_columns = ["cell_id", "frame_start", "t_start_min", "frame_end",
                  "t_end_min", "dx_um", "dy_um", "tau_frames", "tau_min"]
return (steps[output_columns].sort_values(["cell_id", "frame_start"])
        .reset_index(drop=True))

def compute_steps_multi_tau(tracks: pd.DataFrame, taus: list[int]) -> pd.DataFrame:
    validate_tracks(tracks)
    all_steps = []
    for tau in taus:
        all_steps.append(compute_steps_for_tau(tracks, tau_frames=tau))
    return pd.concat(all_steps, ignore_index=True)

def summary_by_tau(steps_all: pd.DataFrame) -> pd.DataFrame:
    return (steps_all.groupby("tau_frames")
            .agg(n_steps=("dx_um", "size"),
                 n_cells=("cell_id", "nunique"),
                 median_tau_min=("tau_min", "median"))
            .reset_index())

```

A12. tests/test_steps.py – scientific unit/regression tests

```

"""Unit tests for displacement-step calculations."""
import pandas as pd
import pytest
from src.steps import compute_steps_for_tau, compute_steps_multi_tau, summary_by_tau

@pytest.fixture
def toy_tracks() -> pd.DataFrame:
    return pd.DataFrame({
        "cell_id": [1,1,1,2,2,2],
        "frame": [0,1,2,0,1,2],
        "t_min": [0.,20.,40.,0.,20.,40.],
        "x_um": [0.,1.,3.,10.,10.,9.],
        "y_um": [0.,2.,2.,5.,6.,8.],
    })

def test_compute_steps_tau1(toy_tracks):
    steps = compute_steps_for_tau(toy_tracks, 1)
    assert steps["dx_um"].tolist() == [1.,2.,0.,-1.]
    assert steps["dy_um"].tolist() == [2.,0.,1.,2.]
    assert steps["tau_min"].tolist() == [20.,20.,20.,20.]

def test_compute_steps_tau2(toy_tracks):
    steps = compute_steps_for_tau(toy_tracks, 2)
    assert steps["frame_end"].tolist() == [2,2]
    assert steps["dx_um"].tolist() == [3.,-1.]
    assert steps["dy_um"].tolist() == [2.,3.]

def test_steps_do_not_cross_cell_boundaries(toy_tracks):
    steps = compute_steps_for_tau(toy_tracks, 1)
    assert steps.groupby("cell_id").size().to_dict() == {1:2, 2:2}

def test_invalid_tau_raises_error(toy_tracks):
    with pytest.raises(ValueError, match="tau_frames must be >= 1"):
        compute_steps_for_tau(toy_tracks, 0)

def test_missing_required_column_raises_error(toy_tracks):
    with pytest.raises(ValueError, match="missing required columns"):
        compute_steps_for_tau(toy_tracks.drop(columns="y_um"), 1)

def test_multi_tau_summary(toy_tracks):
    summary = summary_by_tau(compute_steps_multi_tau(toy_tracks, [1,2]))
    assert summary["n_steps"].tolist() == [4,2]
    assert summary["median_tau_min"].tolist() == [20.,40.]

def test_missing_frame_is_not_mislabeled_as_tau1():
    tracks = pd.DataFrame({
        "cell_id": [1,1], "frame": [0,2], "t_min": [0.,40.],
        "x_um": [0.,3.], "y_um": [0.,1.],
    })
    assert compute_steps_for_tau(tracks, 1).empty
    tau2 = compute_steps_for_tau(tracks, 2)
    assert len(tau2) == 1 and tau2["frame_end"].tolist() == [2]

def test_duplicate_cell_frame_raises_error():
    tracks = pd.DataFrame({
        "cell_id": [1,1], "frame": [0,0], "t_min": [0.,0.],
        "x_um": [1.,1.5], "y_um": [2.,2.5],
    })
    with pytest.raises(ValueError, match="duplicate cell_id-frame observations"):
        compute_steps_for_tau(tracks, 1)

```

A13. Notebook 04 – core Phase 4 analysis code

```

# Notebook 04 – core Phase 4 sequence (condensed from committed executed notebook)
import _path
from _path import PROJECT_ROOT
import numpy as np
import pandas as pd
from src.steps import compute_steps_multi_tau, summary_by_tau, validate_tracks

TRACKS_PATH = PROJECT_ROOT / "data" / "MSC01_tracks_clean.csv"
tracks = pd.read_csv(TRACKS_PATH)
validate_tracks(tracks)

# Temporal-gap audit
tracks_sorted = tracks.sort_values(["cell_id", "frame"]).reset_index(drop=True).copy()
g = tracks_sorted.groupby("cell_id", sort=False)
tracks_sorted["frame_gap"] = g["frame"].diff().astype("Int64")
tracks_sorted["time_gap_min"] = g["t_min"].diff()

```

```
# Rebuild exact lags
steps_exact = compute_steps_multi_tau(tracks, taus=[1,2,4]).copy()
steps_exact["actual_frame_gap"] = steps_exact["frame_end"] - steps_exact["frame_start"]
steps_exact["lag_matches_label"] = steps_exact["actual_frame_gap"] == steps_exact["tau_frames"]
assert steps_exact["lag_matches_label"].all()

# Exact  $\tau=1$  frame-wise population motion
taul_steps = steps_exact.loc[steps_exact["tau_frames"].eq(1)].copy()
def median_absolute_deviation(values: pd.Series) -> float:
    center = values.median()
    return float((values - center).abs().median())

framework_motion = (taul_steps.groupby(["frame_start", "frame_end"])
    .agg(n_cells=("cell_id", "nunique"),
        mean_dx_um=("dx_um", "mean"), mean_dy_um=("dy_um", "mean"),
        median_dx_um=("dx_um", "median"), median_dy_um=("dy_um", "median"),
        mad_dx_um=("dx_um", median_absolute_deviation),
        mad_dy_um=("dy_um", median_absolute_deviation))
    .reset_index())
framework_motion["common_magnitude_um"] = np.hypot(framework_motion["median_dx_um"], framework_motion["median_dy_um"])
framework_motion["cumulative_median_dx_um"] = framework_motion["median_dx_um"].cumsum()
framework_motion["cumulative_median_dy_um"] = framework_motion["median_dy_um"].cumsum()

# Build cumulative correction and co-moving coordinates
frame_correction = pd.DataFrame({"frame": [int(tracks["frame"].min())],
    "common_x_um": [0.0], "common_y_um": [0.0]})
cumulative_part = pd.DataFrame({"frame": framework_motion["frame_end"],
    "common_x_um": framework_motion["median_dx_um"].cumsum(),
    "common_y_um": framework_motion["median_dy_um"].cumsum()})
frame_correction = pd.concat([frame_correction, cumulative_part], ignore_index=True)
tracks_comoving = tracks.merge(frame_correction, on="frame", how="left", validate="many_to_one")
tracks_comoving["x_um_raw"] = tracks_comoving["x_um"]
tracks_comoving["y_um_raw"] = tracks_comoving["y_um"]
tracks_comoving["x_um"] -= tracks_comoving["common_x_um"]
tracks_comoving["y_um"] -= tracks_comoving["common_y_um"]

# Recompute exact lags after coordinate transform
steps_comoving = compute_steps_multi_tau(
    tracks_comoving[["cell_id", "frame", "t_min", "x_um", "y_um"]],
    taus=[1,2,4])

# Final invariant checks
expected_counts = {1:433, 2:398, 4:343}
observed_counts = steps_comoving.groupby("tau_frames").size().to_dict()
assert observed_counts == expected_counts
assert tracks_comoving[["common_x_um", "common_y_um"]].isna().sum().sum() == 0
assert ((steps_comoving["frame_end"] - steps_comoving["frame_start"]) == steps_comoving["tau_frames"]).all()

tracks_comoving.to_csv(PROJECT_ROOT/"data"/"MSC01_tracks_comoving.csv", index=False)
steps_comoving.to_csv(PROJECT_ROOT/"data"/"MSC01_steps_comoving_tau124.csv", index=False)
```

[↑ Back to Contents](#)

Appendix B. Data Dictionaries and Artifact Ledger

Artifact	Role	Public GitHub status	Confidence/status
data/raw/Fluo-C2DL-MSC/01/MSC01_allspots.csv	Raw TrackMate Spots export	Ignored/local	Project input; not versioned
data/MSC01_tracks_clean.csv	Canonical cleaned raw-coordinate tracks	Ignored/local	474x5; 35 tracks; checksum recorded previously
data/MSC01_tracks_clean_min10.csv	≥10-frame sensitivity subset	Ignored/local	18 tracks; later robustness input
data/MSC01_steps_tau124.csv	Combined exact raw steps	Ignored/local	1,174 rows; $\tau=1/2/4$
data/MSC01_tracks_comoving.csv	Co-moving coordinates	Ignored/local	Phase 4 output; validated
data/MSC01_steps_comoving_tau124.csv	Exact steps from co-moving coordinates	Ignored/local	Phase 4 output; counts 433/398/343
notebooks/02_clean_spots_MSC01.ipynb	Cleaning workflow	Tracked	Historical executed notebook
notebooks/03_compute_steps_MSC01.ipynb	Exact-lag Phase 3 workflow	Tracked	Corrected baseline
notebooks/04_drift_validation_MSC01.ipynb	Temporal-gap/common-motion/co-moving workflow	Tracked	Executed; final assertions passed
src/steps.py	Exact-lag reusable logic	Tracked	Repository source of truth
tests/test_steps.py	Scientific unit/regression tests	Tracked	8 tests; total suite 9 passed
README.md	Concise repository landing page	Tracked	Updated through Phase 4 at e0e0909
instructions/...Master_Handbook.pdf	Detailed scientific/project record	Tracked	This updated document should replace the prior Phase 3 copy

[↑ Back to Contents](#)

Appendix C. Python and Biophysics Glossary

Term	Project-specific meaning
Trajectory	A time-ordered series of positions for one tracked cell.
Displacement	A vector difference between two positions.

Term	Project-specific meaning
Lag τ	The temporal separation used to define a displacement or sequence statistic.
Drift	Common apparent motion caused by imaging hardware, registration, sample motion, or other global effects; must be distinguished from collective migration.
Detailed balance	An equilibrium condition relating forward and reverse transitions.
Time irreversibility	Statistical distinguishability of a time-ordered process from its correctly defined time reversal.
Entropy production	A thermodynamic measure of irreversibility; trajectory-level probability ratios can bound or represent it only under explicit models and assumptions.
DataFrame	A pandas table with labeled rows and columns.
Series	A one-dimensional labeled pandas object, usually a column.
Index	Labels used to align pandas objects.
dtype	The stored data type of a Series or array.
NaN	A missing numerical value marker.
list	Ordered mutable collection.
tuple	Ordered immutable collection.
set	Unordered collection of unique hashable elements.
dictionary	Mapping from keys to values.
function	Reusable named block of code with inputs and a return value.
method	A function attached to an object, such as <code>df.sort_values()</code> .
module	A Python <code>.py</code> file.
package	A directory of importable Python modules.
GroupBy	A pandas object supporting split-apply-combine operations.
Vectorization	Operating on arrays/Series at once instead of explicit row loops.
Bootstrap	Resampling procedure used to estimate uncertainty; here it should resample tracks as clusters.
Smoke test	A minimal test that verifies basic import/execution, not scientific correctness.
Unit test	A focused test of a small function against known expected behavior.
Reproducibility	The ability to regenerate outputs from documented code, environment, parameters, and data provenance.
Exact physical lag	A displacement connects <code>frame_start</code> to <code>frame_start + τ_frames</code> for the same cell; row adjacency is not enough.
Common motion	Frame-wise translation shared by the population as estimated by median <code>dx/dy</code> ; physical origin may be technical, biological, or mixed.
Co-moving coordinates	Coordinates after subtracting cumulative frame-wise common motion; retained as a sensitivity-analysis representation.
MAD	Median absolute deviation; robust dispersion statistic around the median.
Regression test	A test added after a discovered bug so the same failure mode cannot silently return.
Cluster bootstrap	Bootstrap that resamples whole cell tracks as dependence-preserving units rather than individual correlated steps.
Sensitivity analysis	Repeat analysis under a defensible alternative preprocessing/model choice to assess conclusion stability.

[↑ Back to Contents](#)

Appendix D. Ready-to-Paste New-Chat Handoff Prompt

Handoff prompt

I am continuing the cell-motility time-irreversibility project.
Treat the attached Master Handbook and the current GitHub main branch as the project record.
The code source of truth is main commit `e0e0909` or later.

Current validated state:

- Phase 4 is complete.
- Exact physical lag counts: $\tau_1=433$, $\tau_2=398$, $\tau_4=343$; total=1174.
- `src/steps.py` uses exact same-cell frame matching, not row shift.
- The test suite has 9 passing tests, including missing-frame and duplicate cell-frame regression tests.
- Phase 4 measured a population-shared common-motion component, strongest in negative `x`.
- Do NOT call that component microscope drift without independent evidence.

- Raw and co-moving trajectories/steps are both retained.
- The phase-4-drift branch was merged via PR #1 and deleted; only main remains.
- README is synchronized through Phase 4.

Next task: Phase 5. Implement/test probability-density estimation for raw and co-moving exact steps; compare $p(\Delta)$ with $p(-\Delta)$; bootstrap by cell track; keep single-step asymmetry separate from full sequence-level temporal irreversibility.

Teaching rule: explain Python, mathematics/statistics, biophysics, reproducibility, and Git decisions as the project advances. Do not treat code as copy-paste.

[↑ Back to Contents](#)

Appendix E. References and Source Snapshot

Repository verification note: current main is e0e0909 ("Update README through Phase 4"), commit d0bf5e5 merged PR #1, and the branches page lists only main after deletion of phase-4-drift. README now reports Phase 4 complete and Phase 5 next. The public repository still contains the older handbook PDF until this updated artifact is committed.

[R1] AbolDpirate/cell-irreversibility GitHub repository, main commit e0e0909, inspected 2 August 2026.
<https://github.com/AbolDpirate/cell-irreversibility>

[R2] Cell Tracking Challenge — 2D+Time datasets and Fluo-C2DL-MSD download page.
<https://celltrackingchallenge.net/2d-datasets/>

[R3] Tinevez J-Y et al. TrackMate: An open and extensible platform for single-particle tracking. *Methods*. 2017;115:80–90. <https://pubmed.ncbi.nlm.nih.gov/27713081/>

[R4] TrackMate LAP trackers documentation. <https://imagej.github.io/plugins/trackmate/trackers/lap-trackers>

[R5] Maška M et al. The Cell Tracking Challenge: 10 years of objective benchmarking. *Nature Methods*. 2023.
<https://www.nature.com/articles/s41592-023-01879-y>

[R6] Seifert U. Stochastic thermodynamics, fluctuation theorems and molecular machines. *Reports on Progress in Physics*. 2012;75:126001. <https://arxiv.org/abs/1205.4176>

[R7] Gnesotto FS et al. Broken detailed balance and non-equilibrium dynamics in living systems: a review. *Reports on Progress in Physics*. 2018;81:066601. <https://pubmed.ncbi.nlm.nih.gov/29504517/>

[R8] Roldán É et al. Quantifying entropy production in active fluctuations of the hair-cell bundle from time irreversibility and uncertainty relations. *New Journal of Physics*. 2021;23:083013. <https://doi.org/10.1088/1367-2630/ac0f18>

Internal project sources used: the earlier Master Handbook (preserved as the cumulative base), the Phase 4 project conversation and screenshots, stored notebook outputs, and the live GitHub main branch. Repository-derived code was cross-checked against `src/steps.py`, `tests/test_steps.py`, `README.md`, commit history, branch state, `.gitignore`, and `.gitattributes` on 2 August 2026. Numerical Phase 4 results were taken from executed notebook outputs supplied in the project record.

[↑ Back to Contents](#)

END OF VERIFIED PROJECT RECORD — PHASE 4 VALIDATED • PHASE 5 READY